

YENEPOYA INSTITUTE OF ARTS, COMMERCE AND SCIENCE

A Constituent Unit of Yenepoya (Deemed to be University), Balmatta, Mangalore

Department of Computer Science

Project – Low Level Design

on

Project Title

Data-Driven Risk Profiling for Supply Chain

Student Name: Mohammad Fazil Anwaz

Industry Mentor:

Guided by:

Table of contents	Pg.no
1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
2. Module-Level Design	3
2.1 Module 1 — Library Setup	4
2.2 Module 2 — Data Definition	5
2.3 Module 3 — Risk Calculation Engine	6
2.4 Module 4 — Visualization	7
2.5 Module 5 — Export & Report	7
3. Detailed Data Structures	8
3.1 Weights Dictionary	8
3.2 Supplier DataFrame Schema	8
4. Algorithm Design	9
4.1 Weighted Score Algorithm	9
4.2 Risk Classification Algorithm	9
5. Error Handling	10
6. Technology & Library Reference	10
7. References	11

1. Introduction

This Low Level Design (LLD) document provides detailed technical specifications for the implementation of "Data-Driven Risk Profiling for Supply Chain" (PRJN26-250). It translates the high-level architecture defined in the HLD into specific module designs, data structures, algorithms, and code-level logic.

1.1 Purpose

The purpose of this document is to provide a complete technical blueprint for implementing the weighted risk scoring engine in Python using Pandas, enabling students and developers to implement or replicate the system with clarity and precision.

1.2 Scope

- Detailed design of all 5 notebook modules
- Data structure definitions for weights and supplier DataFrame
- Step-by-step algorithm for weighted score calculation
- Risk classification logic with thresholds
- Error handling and validation mechanisms
- Complete technology and library reference

2. Module-Level Design

2.1 Module 1 — Library Setup

This module initializes the Python environment by importing all required open-source libraries.

Library	Alias	Version	Purpose
pandas	pd	3.0.2	Data manipulation and DataFrame operations
numpy	np	2.4.4	Numerical computations and array operations
matplotlib.pyplot	plt	Latest	Chart and plot generation
seaborn	sns	Latest	Statistical visualization and styling

Code Implementation:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2.2 Module 2 — Data Definition

This module defines the risk factor weights and supplier input data, creating the primary Pandas DataFrame used throughout the system.

Risk Factor	Weight	% of Total	Framework
Vendor_Financial_Health	0.15	15%	NIST CSF — Identify
Third_Party_Access_Control	0.15	15%	ISO 27001 — A.9
Incident_Response_Readiness	0.10	10%	NIST CSF — Respond
Vulnerability_Management	0.15	15%	CIS Controls v8
Delivery_SLA_Performance	0.10	10%	MITRE ATT&CK SC
Compliance_Certification	0.10	10%	ISO 27001 — A.18
Network_Security_Posture	0.15	15%	CIS Controls v8 — IG2
Data_Privacy_Compliance	0.10	10%	ISO 27001 — A.8
TOTAL	1.00	100%	—

2.3 Module 3 — Risk Calculation Engine

This is the core module of the system. It implements the weighted scoring formula to calculate a composite Risk Score for each supplier.

ALGORITHM — Weighted Risk Score Calculation

```
INPUT: DataFrame df with supplier scores, dict weights
OUTPUT: df with Risk_Score and Risk_Level columns
Step 1: Extract factor list → factors = list(weights.keys())
Step 2: For each factor f in factors:
    weighted_score += df[f] * weights[f]
Step 3: df['Risk_Score'] = sum of all weighted_scores
Step 4: Apply classify_risk() to each Risk_Score value
Step 5: Store result in df['Risk_Level']
```

Risk Classification Thresholds:

Risk Level	Score Range	Action Required	Color Code
Low	0.0 – 3.0	Monitor periodically	Green
Medium	3.1 – 5.5	Review quarterly	Yellow
High	5.6 – 7.5	Immediate review required	Orange
Critical	7.6 – 10.0	Escalate immediately	Red

2.4 Module 4 — Visualization

This module generates a two-panel visual dashboard using Matplotlib and Seaborn:

Chart	Type	Library	Purpose
Chart 1	Horizontal Bar Chart	Matplotlib	Supplier risk score comparison
Chart 2	Pie Chart	Matplotlib	Risk factor weight distribution
Chart 3	Grouped Bar Chart	Matplotlib	Risk distribution by supplier tier
Chart 4	Heatmap	Seaborn	Factor score heatmap across 15 suppliers

- Chart colors mapped to risk level: Green (#4CAF50), Orange (#FFA500), Red (#FF6B6B)
- Threshold lines drawn at 5.5 (High) and 7.5 (Critical) on bar chart
- Dashboard saved as PNG file using `plt.savefig()` at 150 DPI
- `plt.show()` renders the chart inline in Jupyter Notebook

2.5 Module 5 — Export & Report

This module calculates individual weighted factor contributions, exports results to CSV, and prints the final summary report.

- Individual weighted scores calculated: `df['Weighted_FactorName'] = df[factor] * weights[factor]`
- Full enriched DataFrame exported: `df.to_csv('risk_profiling_results.csv', index=False)`
- Summary statistics printed: total suppliers, risk level counts, average score, highest/lowest risk

3. Detailed Data Structures

3.1 Weights Dictionary

The weights dictionary maps each risk factor to its assigned weight value. All weights must sum to exactly 1.0:

```
weights = {
    'Vendor_Financial_Health': 0.20,
    'Third_Party_Access_Control': 0.20,
    'Incident_Response_Readiness': 0.15,
    'Vulnerability_Management': 0.20,
    'Delivery_SLA_Performance': 0.15,
    'Compliance_Certification': 0.10
}
# Validation: sum(weights.values()) == 1.0 → True
```

3.2 Supplier DataFrame Schema

The Pandas DataFrame has the following structure after all processing steps:

- Shape: (5 rows, 17 columns) after weighted factor columns are added
- Index: Default integer index (0 to 4)
- Dtypes: Supplier (object), scores (int64), Risk_Score (float64), Risk_Level (object), Weighted_* (float64)
- Memory usage: Approximately 2-3 KB for the sample dataset

4. Algorithm Design

4.1 Weighted Score Algorithm

The weighted scoring algorithm is implemented using Python's built-in `sum()` function with a generator expression for efficiency:

```
# Step 1: Get factor names
factors = list(weights.keys())

# Step 2: Calculate weighted score using vectorized operations
df['Risk_Score'] = sum(
    df[factor] * weights[factor]
    for factor in factors
)

# Example calculation for Supplier A:
# Risk_Score = (7*0.20) + (6*0.20) + (5*0.15) + (8*0.20) + (6*0.15) +
# (9*0.10)
# Risk_Score = 1.40 + 1.20 + 0.75 + 1.60 + 0.90 + 0.90 = 6.75
```

4.2 Risk Classification Algorithm

The `classify_risk()` function maps a numeric score to a categorical risk level:

```
def classify_risk(score):
    if score <= 3.0:
        return 'Low'
    elif score <= 5.5:
        return 'Medium'
    elif score <= 7.5:
        return 'High'
    else:
        return 'Critical'

# Apply to entire column using pandas .apply()
df['Risk_Level'] = df['Risk_Score'].apply(classify_risk)
```

5. Error Handling

Error Scenario	Detection Method	Resolution
Weights don't sum to 1.0	<code>sum(weights.values()) != 1.0</code>	Re-assign weights to sum to exactly 1.0
Missing library import	ImportError raised	Run: <code>pip install library_name</code>
Score out of range (>10)	ValueError check	Ensure all scores are between 1 and 10
CSV path not found	OSError raised	Run notebook from correct directory
Kernel disconnected	Jupyter kernel error	Restart kernel and run all cells

6. Technology & Library Reference

Tool / Library	Version Used	License	Official Documentation
Python	3.12.x	PSF License	https://docs.python.org/3.12
Pandas	3.0.2	BSD License	https://pandas.pydata.org/docs
NumPy	2.4.4	BSD License	https://numpy.org/doc
Matplotlib	Latest	PSF License	https://matplotlib.org
Seaborn	Latest	BSD License	https://seaborn.pydata.org
Jupyter Notebook	Latest	BSD License	https://jupyter.org
VS Code	Latest	MIT License	https://code.visualstudio.com

7. References

- NIST Cybersecurity Framework (CSF) v1.1 — <https://www.nist.gov/cyberframework>
 - ISO/IEC 27001:2022 — Information Security Management Systems
 - CIS Controls v8 — Center for Internet Security — <https://www.cisecurity.org>
 - MITRE ATT&CK for Supply Chain — <https://attack.mitre.org>
 - Python 3.12 Documentation — <https://docs.python.org/3.12>
 - Pandas Documentation — <https://pandas.pydata.org/docs>
 - Matplotlib Documentation — <https://matplotlib.org/stable/contents.html>
- Seaborn Documentation — <https://seaborn.pydata.org>